

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и безопасность блокчейна»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«ImmutableLog»

Выполнил:

Бардышев Артём Антонович,

студент группы N3346



(подпись)

Суханкулиев Мухаммет,

студент группы N3346



(подпись)

Шангина Елизавета Кирилловна,

студентка группы N3346



(подпись)

Проверил:

Федоров Иван Романович,

преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Введение	4
1 Обзор предметной области и обоснование блокчейна.....	6
1.1 Хэширование и привязка к моменту фиксации	6
1.2 Смарт-контракт как реестр отпечатков	6
1.3 Сопоставление с централизованной базой данных	6
2 Модель нарушителя и требования безопасности	7
2.1 Активы	7
2.2 Угрозы	7
2.3 Принятые меры	7
3 Архитектура системы.....	9
3.1 Логические компоненты	9
3.2 Потоки данных.....	9
4 Реализация смарт-контракта.....	10
4.1 Язык и зависимости.....	10
4.2 Структура данных и интерфейс	10
4.3 Безопасность и инварианты.....	10
5 Прикладной модуль: хэширование, Web3 и интерфейс	11
5.1 Хэширование.....	11
5.2 Web3 и конфигурация	11
5.3 Интерфейс Streamlit.....	11
5.4 Сообщения об ошибках доступа	11
6 Сценарии использования и демонстрация работы	13
6.1 Сценарий регистрации критически значимых событий и успешной записи в реестр 13	
6.2 Сценарий просмотра реестра в боковой панели.....	13
6.3 Сценарий аудита неизменённого журнала.....	14
6.4 Сценарий выявления компрометации журнала (файл `compromised_log.txt`)	15
6.5 Сценарий выявления компрометации журнала (файл `ip_log.txt`, два фрагмента) 15	
7 Инфраструктура развёртывания и тестирование.....	16
7.1 Docker Compose	16
7.2 Локальный блокчейн и развёртывание контракта.....	16

7.3	Тестирование.....	16
8	Результаты и эффект от внедрения	17
8.1	Достигнутые свойства.....	17
8.2	Значение для практики информационной безопасности	17
	Заключение.....	18

ВВЕДЕНИЕ

Журналы событий (логи) операционных систем и прикладного программного обеспечения, как правило, размещаются на том же вычислительном узле, который обслуживает соответствующие сервисы. В случае компрометации учётной записи с административными привилегиями возникает риск не только сокрытия следов несанкционированной активности, но и модификации либо удаления записей в локальных файлах журналов. Традиционные организационно-технические меры (централизованный сбор логов, носители с однократной записью, системы класса SIEM) предполагают наличие выделенной инфраструктуры и определённого уровня доверия к оператору централизованного хранения. Применение распределённого реестра с механизмом консенсуса позволяет зафиксировать факт существования криптографического отпечатка данных к моменту включения соответствующей транзакции в цепочку блоков; последующее изменение экземпляра журнала на узле-источнике не устраняет ранее зафиксированное состояние реестра.

Объектом исследования являются процессы сбора, хранения и аудита журналов событий в контексте информационной безопасности. Предметом исследования выступает применение смарт-контракта и программного веб-клиента для регистрации хэшей отобранных строк журнала и для процедур проверки целостности файла журнала со стороны лица, осуществляющего аудит.

Целью настоящей работы является разработка и описание прототипа программного комплекса, демонстрирующего возможность использования блокчейна в качестве внешнего по отношению к узлу с журналом аппенд-онли реестра отпечатков критически значимых событий при ограничении права записи уполномоченным субъектом и при наличии средств обнаружения удаления строк из локальной копии журнала после регистрации отпечатков.

Для достижения указанной цели последовательно решаются задачи анализа типовой модели угроз к журналам на узле и обоснования целесообразности выноса якоря целостности в смарт-контракт; проектирования архитектуры взаимодействия узла блокчейна, смарт-контракта, серверной логики на языке Python и веб-интерфейса; реализации смарт-контракта с хранением массива записей «хэш — временная метка», с контролем доступа к функции записи и с дополнительными ограничениями на частоту вызовов; реализации вычисления хэшей SHA-256 для отобранных строк журнала и формирования транзакций посредством библиотеки Web3; реализации веб-интерфейса для

регистрации отпечатков, синхронизации с состоянием контракта и проведения проверки целостности загружаемого файла; обеспечения воспроизводимости развёртывания посредством контейнеризации, конфигурирования через переменные окружения и сопровождения репозитория документацией; подготовки иллюстративного материала, подтверждающего работоспособность ключевых сценариев (каталог `docs/screenshots/`).

Настоящий отчёт излагает постановку задачи, теоретическое обоснование применения блокчейна, модель нарушителя, архитектуру и реализацию компонентов, порядок развёртывания и испытаний, а также формулирует выводы по результатам работы.

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ БЛОКЧЕЙНА

1.1 Хэширование и привязка к моменту фиксации

Криптографическая хэш-функция SHA-256 отображает входные данные произвольной длины в строку фиксированной длины. Для целей настоящего прототипа принимается практическая криптографическая стойкость функции в части поиска второго прообраза. Включение значения хэша в состав транзакции, принятой сетью, связывает отпечаток с моментом упорядочивания транзакций в реестре и с временными характеристиками блока, что может использоваться при анализе инцидентов как аргумент в пользу существования определённого отпечатка к рассматриваемому моменту времени.

1.2 Смарт-контракт как реестр отпечатков

Смарт-контракт обеспечивает хранение упорядоченного набора структур вида «строковое представление хэша — временная метка», генерацию журнальных событий (например, `HashRegistered`) для внешних наблюдателей и программный интерфейс чтения состояния. Исторические транзакции в рамках действующей цепочки не подменяются узлами сети без смены канона цепочки (форка). В учебной конфигурации используется локальный клиент сети (Ganache); для публичной сети модификация прошлой истории сопряжена с существенно большими затратами по сравнению с правкой одного файла на скомпрометированном хосте.

1.3 Сопоставление с централизованной базой данных

Централизованная база данных, размещаемая в периметре организации, в общем случае подвержена тем же классам угроз, что и файлы журналов на администрируемых узлах (компрометация учётных записей администраторов, несанкционированное изменение резервных копий). Размещение якоря целостности в распределённом реестре выносит критический элемент за пределы одного хоста и иллюстрирует принцип разделения полномочий между источником журнала и хранилищем отпечатков.

2 МОДЕЛЬ НАРУШИТЕЛЯ И ТРЕБОВАНИЯ БЕЗОПАСНОСТИ

2.1 Активы

- целостность журнала: соответствие текущего содержимого файла множеству событий, считающихся зафиксированными в ходе регистрации;
- конфиденциальность текста отдельных строк: в распределённый реестр по замыслу прототипа передаются хэши; полный текст критически значимых строк дублируется в локальном хранилище у агента или аудитора, что образует дополнительную поверхность атаки;
- полномочия на запись в смарт-контракт: право инициирования новых записей в реестре.

2.2 Угрозы

1. удаление или подмена строк в локальном файле журнала после инцидента;
2. несанкционированная массовая запись в смарт-контракт с произвольных адресов (в том числе с целью заполнения реестра недостоверными отпечатками);
3. компрометация ключа уполномоченного агента, приводящая к подписанию транзакций, корректных с точки зрения контракта, но не соответствующих фактической политике организации.

2.3 Принятые меры

Угроза	Мера в прототипе
Подмена журнала на узле	сравнение множества хэшей отобранных строк с эталоном, сформированным при регистрации; вывод сведений о «восстановленных фрагментах»
Несанкционированная запись в контракт	разграничение доступа: вызов функции записи разрешён только владельцу контракта (реализация на базе контракта Ownable библиотеки OpenZeppelin); отказ в исполнении при вызове с неавторизованного адреса на уровне виртуальной машины Ethereum

Высокая частота записи от имени владельца	интервал между успешными вызовами функции записи (кулдаун), задаваемый логикой смарт-контракта
Наблюдаемость состояния реестра	события `HashRegistered`, чтение массива записей через прикладной интерфейс (режим синхронизации с сетью)

Для снижения вероятности несанкционированной записи в реестр со стороны произвольных участников сети реализован механизм авторизации: функция `registerHash` доступна только уполномоченному адресу владельца. Попытка записи с адреса, не обладающего данным полномочием, завершается отказом исполнения на уровне виртуальной машины (в том числе с кодом ошибки, соответствующим нарушению политики Ownable).

3 АРХИТЕКТУРА СИСТЕМЫ

3.1 Логические компоненты

1. узел блокчейна (в учебной конфигурации — Ganache): исполнение транзакций и хранение состояния смарт-контракта;
2. смарт-контракт ImmutableLog: хранение массива записей аудита, контроль доступа, ограничение частоты записи;
3. прикладной модуль на языке Python: вычисление хэшей SHA-256 для отобранных строк, формирование и подпись транзакций посредством библиотеки Web3.py с использованием ключа владельца из конфигурации окружения, абстракция реального и имитационного клиента блокчейна для тестирования;
4. веб-интерфейс на базе Streamlit: разделы регистрации событий, аудита инцидентов, боковая панель реестра блокчейна;
5. локальный файл shadow_db.json: отображение «значение хэша — исходная строка» для восстановления текста удалённых при проверке событий.

3.2 Поток данных

1. регистрация: текстовый файл → разбор строк → отбор критически значимых → вычисление хэша → транзакция вызова `registerHash` → фиксация в реестре; параллельно обновляется локальное хранилище соответствий;
2. синхронизация реестра: чтение числа записей и обращение к элементам массива `auditTrail(i)` → отображение таблицы «хэш — дата и время»;
3. аудит: файл для проверки → множество хэшей отобранных строк → сравнение с множеством ключей локального хранилища → вывод заключения о целостности либо о выявленных удалениях и перечень восстановленных фрагментов.

4 РЕАЛИЗАЦИЯ СМАРТ-КОНТРАКТА

4.1 Язык и зависимости

Смарт-контракт реализован на языке Solidity с использованием компилятора, совместимого с выбранной версией pragma и с библиотекой OpenZeppelin. Для ограничения круга лиц, обладающих правом записи, применяется контракт Ownable.

4.2 Структура данных и интерфейс

- структура LogEntry: строковое поле хэша и целочисленная временная метка блока на момент записи;
- динамический массив записей с модификатором доступа, обеспечивающим публичное чтение по индексу;
- функция `registerHash`: доступна только владельцу; проверяется непустота строки хэша; учитывается минимальный интервал между записями; генерируется событие `HashRegistered`;
- функция `getLogsCount`: возвращает число записей для итерации при чтении со стороны клиента.

4.3 Безопасность и инварианты

Владелец контракта устанавливается при развёртывании (адрес инициатора транзакции создания контракта в конструкторе Ownable). Вызов `registerHash` с адреса, не совпадающего с адресом владельца, приводит к отклонению транзакции. Ограничение частоты записи снижает интенсивность возможных операций при компрометации ключа владельца в рамках постановки прототипа.

5 ПРИКЛАДНОЙ МОДУЛЬ: ХЭШИРОВАНИЕ, WEB3 И ИНТЕРФЕЙС

5.1 Хэширование

Для каждой отобранной строки выполняется нормализация (удаление завершающих пробельных символов), кодирование в UTF-8 и вычисление SHA-256. В состав транзакции передаётся шестнадцатеричное представление отпечатка, что снижает объём раскрываемой в цепочке информации по сравнению с передачей полного текста строки.

5.2 Web3 и конфигурация

Параметры подключения задаются переменными окружения: ``WEB3_RPC_URL``, ``CONTRACT_ADDRESS``, ``OWNER_PRIVATE_KEY``, при необходимости ``WEB3_CHAIN_ID``. Последовательность операций включает установление соединения с провайдером, сборку транзакции вызова ``registerHash``, криптографическую подпись и отправку сырой транзакции в сеть с ожиданием квитанции о включении в блок.

5.3 Интерфейс Streamlit

Боковая панель содержит элементы навигации к реестру и кнопку синхронизации с сетью; основная область разделена на вкладки регистрации событий и аудита инцидентов. Визуальное оформление (тёмная цветовая схема, выделение информационных блоков) ориентировано на задачи оперативного просмотра результатов проверки.

5.4 Сообщения об ошибках доступа

При использовании ключа, не соответствующего адресу владельца контракта, прикладной уровень отображает сообщение о запрете операции записи и указывает причину отказа, в частности код ``OwnableUnauthorizedAccount``, что наглядно демонстрирует принудительное применение политики доступа на уровне исполнения смарт-контракта.

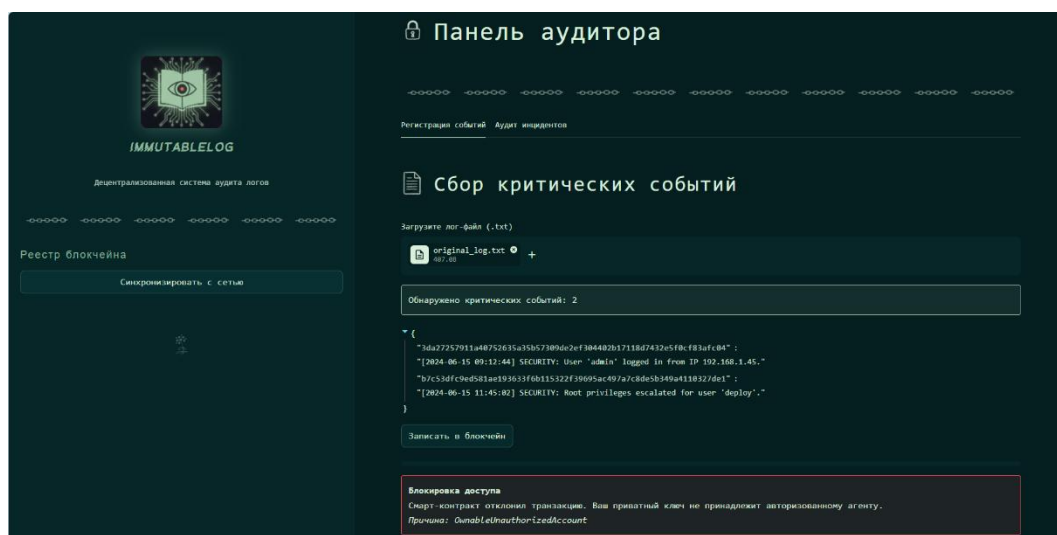


Рисунок 1 - Отказ в записи при использовании неавторизованного ключа:
сообщение интерфейса о блокировке доступа.

6 СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ И ДЕМОНСТРАЦИЯ РАБОТЫ

6.1 Сценарий регистрации критически значимых событий и успешной записи в реестр

Оператор загружает файл `original\_log.txt`. Система определяет число критически значимых событий, отображает пары «хэш — текст строки». По команде записи отпечатки регистрируются в смарт-контракте от имени владельца; интерфейс фиксирует успешное завершение цикла операций.

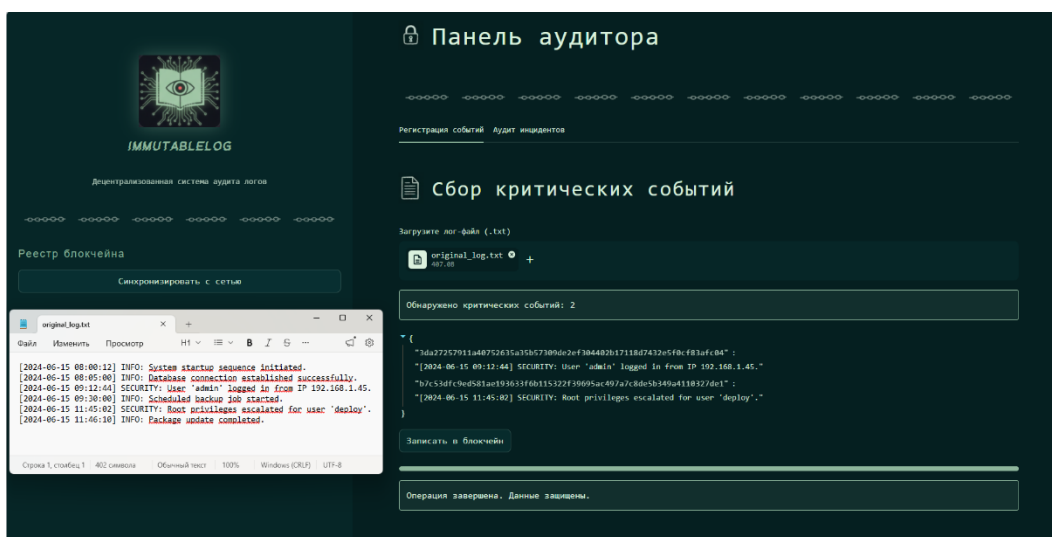


Рисунок 2 - Регистрация событий: две записи категории SECURITY, отображение хэшей и исходных строк, индикация завершения записи в блокчейн.

6.2 Сценарий просмотра реестра в боковой панели

Пользователь инициирует синхронизацию с сетью и получает таблицу отпечатков и временных меток по данным смарт-контракта, что подтверждает корректность чтения состояния реестра клиентским приложением.

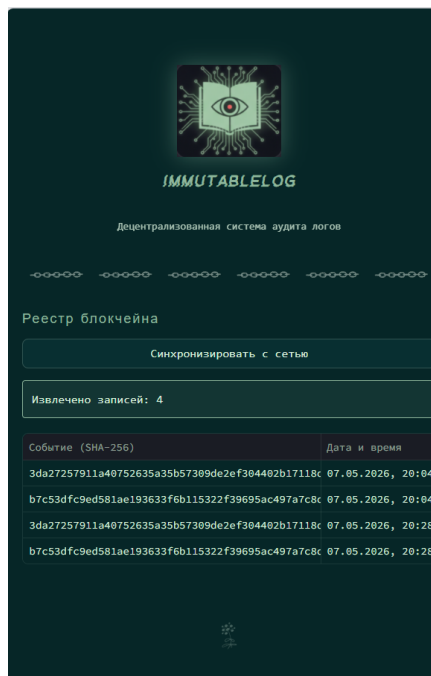


Рисунок 3 - Регистр блокчейна: таблица хэшей и дат после синхронизации.

6.3 Сценарий аудита неизменённого журнала

Загружается файл `original\_log.txt`, состав критически значимых строк в котором соответствует зарегистрированному ранее. Результатом проверки является сообщение о подтверждении целостности.

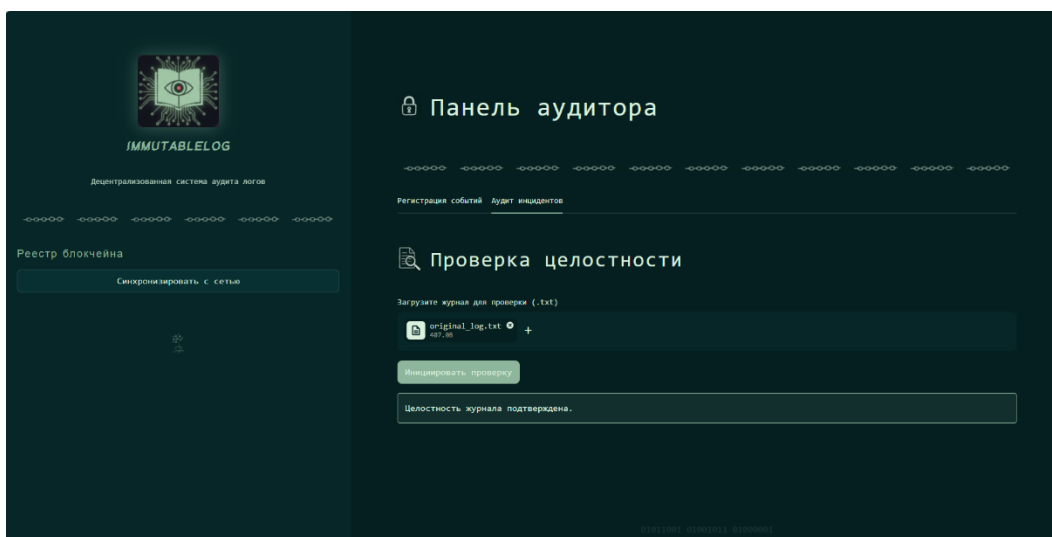


Рисунок 4 - Аудит инцидентов: подтверждение целостности для файла `original\_log.txt`

6.4 Сценарий выявления компрометации журнала (файл `compromised\_log.txt`)

Загружается журнал с удалённой частью критически значимых строк. Система формирует предупреждение о компрометации и выводит восстановленные по локальному хранилищу фрагменты с указанием хэш-доказательства.

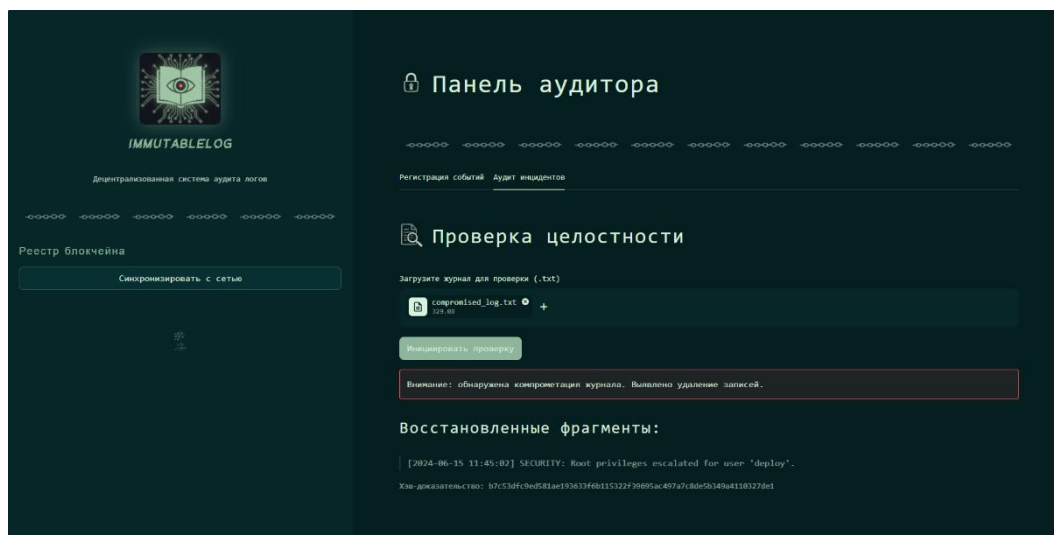


Рисунок 5 - Выявление удаления записей; восстановление не менее одного фрагмента (эскалация привилегий для учётной записи `deploy`).

6.5 Сценарий выявления компрометации журнала (файл `ip\_log.txt`, два фрагмента)

Дополнительная иллюстрация демонстрирует ту же процедуру для файла `ip\_log.txt` при восстановлении двух строк с отдельными хэш-доказательствами.

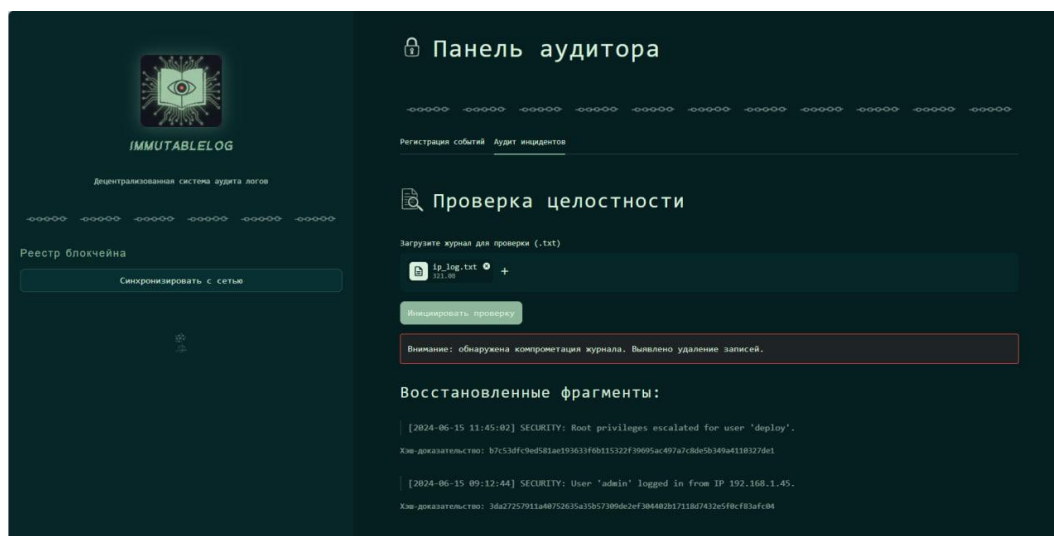


Рисунок 6 - Два восстановленных фрагмента и соответствующие хэши.

7 ИНФРАСТРУКТУРА РАЗВЁРТЫВАНИЯ И ТЕСТИРОВАНИЕ

7.1 Docker Compose

Прикладной компонент размещается в контейнере; в конфигурации Docker Compose предусмотрено монтирование исходного кода, каталога с описанием ABI контракта и, при необходимости, тома для файла ``shadow_db.json``. Секретные параметры задаются через файл `` .env ``, исключённый из системы контроля версий.

7.2 Локальный блокчейн и развёртывание контракта

Для воспроизведения эксперимента рекомендуется использование узла Ganache и развёртывание смарт-контракта средой Remix с последующим переносом адреса контракта и ABI в файлы проекта. Адрес учётной записи, инициировавшей создание контракта, становится владельцем.

7.3 Тестирование

Предусмотрены модульные и сценарные проверки средствами Pytest (в том числе проверка вычисления хэша, поведение имитации блокчейна, сценарии интерфейса Streamlit). Для контроля качества исходного кода используются статические анализаторы Ruff и Муру в соответствии с настройками репозитория.

8 РЕЗУЛЬТАТЫ И ЭФФЕКТ ОТ ВНЕДРЕНИЯ

8.1 Достигнутые свойства

1. фиксация отпечатков критически значимых событий в распределённом реестре с привязкой к моменту включения транзакций;
2. программно-аппаратно реализуемое ограничение права записи на уровне смарт-контракта;
3. экспериментально воспроизводимая демонстрация обнаружения удаления строк из журнала и восстановления текста по локальному хранилищу;
4. документированная и контейнеризованная процедура развёртывания прототипа.

8.2 Значение для практики информационной безопасности

Прототип иллюстрирует направление усиления доказуемости целостности журналов за счёт выноса якоря отпечатков за пределы одного узла, формализации политики доступа к операции записи в реестр и предоставления средств интерактивной проверки для лица, осуществляющего аудит. Перенос архитектуры на корпоративный приватный блокчейн или сеть второго уровня представляет собой логическое продолжение исследования за рамками учебного стенда.

ЗАКЛЮЧЕНИЕ

В работе выполнен цикл проектирования и реализации программного комплекса ImmutableLog, объединяющего смарт-контракт на языке Solidity с библиотекой OpenZeppelin, серверную логику на языке Python с использованием Web3 и веб-интерфейс на платформе Streamlit. Реализована регистрация отпечатков SHA-256 для отобранных строк журналов при разграничении доступа к записи в контракт и реализованы процедуры проверки целостности загружаемого файла журнала. Экспериментально подтверждены сценарии успешной регистрации и чтения реестра, отказа в записи при неавторизованном ключе и выявления компрометации журнала с отображением восстановленных фрагментов.

Результаты соответствуют учебной постановке дисциплины: продемонстрированы понимание архитектуры распределённого реестра, модели угроз и обоснование применения блокчейна в задаче журналирования и аудита. Перспективы развития включают подключение постоянного агента сбора на узле-источнике, централизованное управление секретами ключей, расширение модели ролей и отдельный анализ экономики транзакций для выбранной среды исполнения.